

SM2_EXAMEN_VALIDACIONES

Informacion General

- Curso: Desarrollo de Aplicaciones Moviles
- Alumno: Javier
- Codigo de estudiante: [Completar codigo]
- Fecha: 02/06/2026
- Repositorio publico: https://github.com/javierpool/SM2_EXAMEN_VALIDACIONES

Detalle de la Implementacion

La implementacion se realizo en la pantalla de registro de la aplicacion SafeArea, ubicada en `lib/screens/register_screen.dart`. Esta pantalla mantiene el flujo original de creacion de cuenta y ahora valida los datos ingresados con widgets nativos de Flutter:

- Form
- GlobalKey<FormState>
- TextFormField

El formulario ejecuta las validaciones unicamente al presionar el boton Crear Cuenta, mediante:

```
_formKey.currentState!.validate()
```

Los errores se muestran como texto nativo debajo de cada campo, sin usar cuadros de dialogo para los errores de formato.

Validaciones RegExp Implementadas

Correo electronico

Campo: Correo electronico

Expresion regular usada:

```
RegExp(r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
```

Esta regla exige un formato estandar de correo, por ejemplo `usuario@dominio.com`, validando usuario, simbolo @, dominio y extension.

Contrasena

Campo: Contraseña

Expresión regular usada:

```
RegExp('^(?=.*[a-z])(?=.*[A-Z])(?=.*\d){8,}$')
```

Esta regla exige:

- Mínimo 8 caracteres.
- Al menos una letra mayúscula.
- Al menos una letra minúscula.
- Al menos un número.

Celular

Campo: Celular (opcional)

Expresión regular usada:

```
RegExp('^[0-9]{9}$')
```

Esta regla permite solo números y valida una longitud exacta de 9 dígitos, usada para celulares en Perú.

Experiencia de Usuario

Cada campo usa un teclado virtual adecuado:

- Nombre completo: `TextInputType.name`
- Correo electrónico: `TextInputType.emailAddress`
- Contraseña: `TextInputType.visiblePassword`
- Celular: `TextInputType.number`

El campo de contraseña oculta los caracteres con `obscureText` y cuenta con un botón de ojo para alternar la visibilidad.

Cuando la validación es correcta, el botón `Crear Cuenta` se deshabilita y muestra un `CircularProgressIndicator` durante 2 segundos para simular el procesamiento del registro.

Evidencias de Funcionamiento

Captura 1: Formulario con datos erróneos

Pegar aqui una captura del formulario despues de presionar Crear Cuenta con campos vacios o invalidos. Deben observarse los mensajes nativos de error debajo de los campos.

Archivo sugerido:

`docs/evidencias/captura_1_validaciones.png`

Captura 2: Formulario con datos correctos y boton cargando

Pegar aqui una captura del formulario con datos validos y el boton en estado de carga o deshabilitado durante la simulacion de envio.

Archivo sugerido:

`docs/evidencias/captura_2_carga.png`

Archivos Modificados

- `lib/screens/register_screen.dart`
- `lib/widgets/auth_text_field.dart`
- `.gitignore`

Verificacion Local

La aplicacion fue compilada en modo web con:

```
flutter build web --release --no-pub
```

Resultado:

```
Built build\web
```